

A Threshold-Based Full-Decentralized Authentication and Key Agreement Scheme for VANETs Powered by Consortium Blockchain

Lu Wei , Yongjuan Zhang, Jie Cui , Senior Member, IEEE, Hong Zhong , Irina Bolodurina , and Debiao He 

Abstract—The authentication and key agreement (AKA) scheme for VANETs can produce a series of short-term session keys, which can be used to secure the vehicular communications across open and insecure wireless channels. Traditional VANETs AKA schemes tend to employ the centralized trust architecture as the core authentication backend, which raises concerns about system security and reliability. Recently, several VANETs AKA schemes that are constructed on decentralized trust architecture have been proposed. However, these schemes do not achieve full decentralization and tend to suffer from key exposure issues, insufficient performance, and lack of optimization for on-chain storage costs. To address these shortcomings, we propose a threshold-based full-decentralized VANETs AKA scheme that is powered by consortium blockchain. In our proposed scheme, the threshold-based voting concept is employed to mitigate the key exposure issue inherent to the network infrastructure. Furthermore, we leverage lightweight cryptography in conjunction with the Cuckoo filter to reduce computational, communication, and on-chain operation costs brought by cryptographic operations and smart contracts. The security proof together with the cryptographic protocol validation tool prove the security of our proposed scheme, whereas the simulation experiment demonstrates the efficiency of our proposed scheme.

Index Terms—VANETs, authentication and key agreement, threshold, consortium blockchain, full decentralization.

Manuscript received 30 November 2023; revised 6 March 2024; accepted 5 June 2024. Date of publication 11 June 2024; date of current version 5 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62302008, Grant U23A20308, Grant 62202008, and Grant 62325209, in part by the Fundamental Research Funds for the Central Universities under Grant 2042023KF0203, in part by the Natural Science Foundation of Anhui Province, China under Grant 2208085QF196, and in part by the University Synergy Innovation Program of Anhui Province under Grant GXXT-2022-049. Recommended for acceptance by J.S. Sun. (Corresponding author: Jie Cui.)

Lu Wei, Yongjuan Zhang, Jie Cui, and Hong Zhong are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230601, China, also with the Anhui Engineering Laboratory of IoT Security Technologies, Anhui University, Hefei 230039, China, and also with the Institute of Physical Science and Information Technology, Anhui University, Hefei 230601, China (e-mail: cuijie@mail.ustc.edu.cn).

Irina Bolodurina is with the Faculty of Mathematics and Information Technologies, Orenburg State University, 460018 Orenburg, Russia (e-mail: prmat@mail.osu.ru).

Debiao He is with the School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China, and also with the Shanghai Key Laboratory of Privacy Preserving Computation, MatrixElements Technologies, Shanghai 201204, China (e-mail: hedeibiao@163.com).

Digital Object Identifier 10.1109/TMC.2024.3412106

I. INTRODUCTION

THE vehicular ad-hoc networks (VANETs) represent a critical subsystem within the broader intelligent transportation system (ITS). Empowered by the recent advancements in wireless communication technology, VANETs provide drivers with access to enriched information and services, thereby enhancing both driving comfort and safety. For example, vehicles send beacons or basic safety messages (BSMs) periodically to nearby vehicles or infrastructures, so that the real state of vehicles can be perceived beyond the line of sight.

In VANETs, a majority of entities, primarily vehicles and roadside units (RSUs), interact via public channels. The inherent openness of these channels leaves them insecure, increasing their susceptibility to a diverse range of security attacks and threats. To ensure the security of vehicular communications, the crafting of an authentication and key agreement (AKA) scheme specifically for VANETs becomes a paramount concern. The AKA scheme for VANETs would generate session keys, which when combined with off-the-shelf encryption and authentication techniques, could facilitate the establishment of secure communication channels.

Various centralized AKA schemes have been proposed for securing vehicular communications, in which the centralized trusted authority is always required to provide the global authentication service for the communication entities within VANETs. Vijayakumar et al. [1] proposed a batch authentication-based key agreement scheme, which employs a batch authentication algorithm executed by an RSU. However, the incorporation of bilinear pairs results in substantial computational and communication costs for the scheme. Dua et al. [2] proposed a two-tier certifiable key agreement scheme in which the cluster head vehicle is directly responsible for AKA with surrounding vehicles after completing the authentication with the trusted center. Nevertheless, the scheme's effectiveness hinges on the assumption of the cluster head vehicle's honesty and trustworthiness. Saleem et al. [3] proposed a lightweight and secure privacy-preserving AKA scheme. This scheme delivers an efficient and secure data transfer mechanism over a public communication channel. However, the scheme's use of hash functions results in a computational cost that remains undesirably high.

The configuration of a centralized trusted authority is vulnerable to attacks from malicious adversaries, which could render authentication services inaccessible to vehicles. In the most

severe scenarios, secret key information pertaining to vehicles may also be compromised. In light of these advantages, multiple VANETs AKA schemes based on a decentralized trust architecture have been recently proposed. Wei et al. [4] put forth an AKA scheme predicated on a multi-trusted entity model. However, while this scheme takes into account the robustness of the trusted central end from a distributed perspective, it neglects the inner corporation or consensus of the proposed multi-trusted entity system. Li et al. [5] proposed an unlinkable AKA scheme for vehicle-to-vehicle (V2V) communication, based on blockchain technology. This scheme achieves authentication without the need to query vehicle information and assures unlinkability during the V2V phase. However, its heavy reliance on anonymous authentication credentials stored in the blockchain results in reduced authentication efficiency. Vangala et al. [6] proposed an efficient AKA scheme, built on both blockchain technology and the elliptic curve method. While this scheme exhibits lower computational costs and enhanced security, it necessitates comparable communication costs to achieve an increased level of security and functional features.

While state-of-the-art AKA schemes based on the decentralized trust architecture exhibit superior performance in terms of system security, reliability, and privacy compared to those based on the centralized trust architecture, they also exhibit certain deficiencies. The first shortcoming is that these schemes do not fully achieve decentralization. Despite employing a decentralized trust architecture, the authentication service of the AKA process still necessitates the participation of the specific trusted authority. The second issue pertains to key exposure, a common problem encountered in these schemes. Specifically, these schemes utilize identity-based cryptography, which involves a master secret key (held by trusted authorities) to implement the authentication algorithm. This setup results in a situation where the leakage of the master secret key compromises the secret keys of the users. Lastly, the performance of these blockchain-based schemes is often not efficient enough, particularly with regard to the on-chain-related gas costs associated with storing public keys or vehicle credentials.

To address these aforementioned issues, we propose a threshold-based full-decentralized AKA scheme for VANETs powered by consortium blockchain. Specifically, a certain number of edge servers (ESs), which are semi-trusted entities within VANETs and have the completely equal and independent relationship, are connected to construct an authentication network via the consortium blockchain. We design a threshold-based AKA scheme that is compatible with the authentication network, so the AKA message between the vehicle and RSU can be trusted, provided that the number of tokens issued by ESs for validating the trustworthiness of the AKA message meets the predefined threshold. Additionally, we designed the smart contract function based on the Cuckoo filter, which is a typical probabilistic data structure, to greatly reduce the on-chain storage costs, thus improving the system performance and reducing the execution costs. The main contributions of our proposed scheme are as shown follows:

- 1) A threshold-based AKA scheme has been designed for securing vehicular communications. In this scheme, semi-trusted ESs are the only trust infrastructure for providing the authentication service, whereas the traditional centralized or decentralized AKA schemes always require the setup of full-trusted infrastructure. Furthermore, our design leverages the concept of threshold-based voting in the AKA scheme, eliminating the need for the master secret key. As a result, the system's security remains unaffected even in the event of a compromise or offline status of some ESs.
- 2) An optimized smart-contract-based distributed authentication mechanism has been designed to guarantee the authenticity of the AKA message based on the Cuckoo filter, where the public key registration, validation, and revocation functions have been implemented with low on-chain storage costs. Compared with the state-of-the-art blockchain-based VANETs AKA schemes that neglect the on-chain storage costs, our proposed scheme delivers superior economic efficiency and availability.
- 3) We use lightweight cryptography to design our proposed scheme for decreasing the computation and communication costs. The experiment results demonstrate that our proposed has the certain performance advantage over the state-of-the-art VANETs AKA schemes.

II. RELATED WORK

The AKA scheme, which is essential for securing vehicular communications, mainly solves two core problems. One is to guarantee the identity of the participants and the authentication of the interactive messages used for key agreement. The other is to ensure the correctness and security of key agreement process. Since the communication channels for VANETs are open and insecure, it is necessary to construct an AKA mechanism to endorse the legitimacy of the participants. The existing VANETs AKA schemes often set up a TA as the legitimacy endorsement mechanism, ensuring the legitimacy of the identity of the communication entity endorsed by the TA. According to the difference in the trusted architectures, the existing VANETs AKA schemes can be separated into two types: the schemes based on the centralized trusted architecture and the schemes based on the decentralized trusted architecture. The relevant research status is summarized as follows.

A. The VANETs AKA Schemes Based on Centralized Trusted Architecture

Most of the existing AKA schemes for VANETs are based on the centralized trusted architecture, which relies on a fully trustworthy center to implement the registration, authentication, revocation, and other functions for the VANETs communication entities.

To achieve secure communication in VANETs, Liu et al. [7] proposed an efficient anonymous two-way AKA scheme, in which vehicles can generate their own anonymous identity and temporary encryption keys. Vijayakumar et al. [1] proposed an AKA scheme based on batch authentication, in which the

usage of the batch authentication algorithm effectively reduces the computational costs of the vehicle. However, these schemes require computationally costly bilinear pairing operations and frequent interactions, resulting in significant storage and communication costs.

To improve the efficiency of secured communication in vehicular networks, Huang et al. [8] proposed an AKA scheme using ECDSA and Diffie-Hellman key exchange algorithms. Wang et al. [9] proposed an anonymous AKA scheme supporting self-authentication, in which the usage of the self-authenticated public key allows for secure and efficient V2I communication. To reduce the load of the TA, Dua et al. [2] proposed a two-layer AKA scheme, in which the cluster head vehicle is directly responsible for the authentication with the surrounding vehicles after completing the authentication with the TA. However, the scheme relies on the assumption that the cluster head vehicle is honest and reliable.

To solve the problem of secure communication between vehicles and cloud servers, Jiang et al. [10] designed a three-factor AKA scheme, taking into account password and biometric changes, as well as vehicle revocation and re-registration, and also solved the problem of vehicle switching when communicating across cloud servers. However, the dependence of the partial interaction steps on the secure channel results in reduced practicality. In order to improve the computational efficiency of the key agreement process, Cui et al. [11] proposed an AKA scheme based on chaotic map. The usage of hash function and chaotic map may make the key agreement process have lower computational costs. These schemes mentioned above use elliptic curve cryptography or chaotic cryptography to design AKA protocols, which avoid the usage of bilinear pairing operations with high costs but still have the problem that the computational and communication costs are not low enough.

To effectively control the computational costs of the key agreement process, Islam et al. [12] proposed a cryptography-based authenticated group key agreement scheme, which constructs a lightweight authentication scheme based on a secure hash function and a long-term session cipher. Cui et al. [13] designed a lightweight authenticated group key agreement scheme based on hash function, which constructs an authentication algorithm based on hash function and a group key distribution algorithm based on the Chinese Remainder Theorem. These schemes mentioned above use hash function to design AKA protocols, which effectively control the computational costs of the schemes, but there are some defects such as insufficient security, impractical security assumptions, high communication costs, and so on.

B. The VANETs AKA Schemes Based on Decentralized Trusted Architecture

Compared with the centralized trusted architecture, the AKA schemes based on the decentralized trusted architecture can effectively improve the robustness and reliability of the system, and make the system has higher security strength to resist distributed denial-of-service attacks, key-stealing attacks, and other security attacks.

To improve the robustness of the system and reduce the dependence on idealized security hardware devices, Zhang et al. [14] proposed a distributed aggregate authentication scheme, in which a hierarchical trusted architecture with a root trusted entity and a lower-level trusted entity as the main body is used. Wei et al. [4] proposed an AKA scheme based on the multi-trusted entity model, in which the root TA is only responsible for the entity registration function and the sub-TA is responsible for directly providing the authentication function for vehicles and fog nodes, providing the reliability and concurrency of the system robustness of the TA from the decentralized perspective, whereas lack consideration for internal cooperation. Since the characteristics of non-tampering, decentralization, and auditability of the blockchain technology, it can be used to realize authentication [15], [16], trust management [17], [18] and other functions in the communication process of VANETs. In recent years, various blockchain-based AKA schemes have been proposed. Ma et al. [19] proposed an efficient decentralized AKA scheme, which can realize the legitimacy authentication of vehicle users in a decentralized environment by using blockchain to construct functions such as registration, update, and revocation of vehicle public keys. However, this scheme can only achieve local decentralization.

Zhang et al. [20] proposed an asymmetric AKA scheme based on blockchain, in which the secure exchange and transmission of information among group members are realized. However, due to the use of bilinear pairings and the delay in the key agreement process, the scheme is difficult to meet the requirements of practical V2X communication scenarios. Meng et al. [21] proposed a mutual AKA scheme based on private chain. In this scheme, the shared keys are recorded in the private chain, so that the vehicle can securely communicate with all edge computing servers by using the same shared key. However, the blockchain that is used to store the shared keys is managed by RSUs, which are vulnerable to security attacks. Li et al. [5] proposed an AKA scheme based on blockchain, which can effectively resist collusion attacks and achieve unlinkability. However, their scheme relies on frequent homomorphic encryption, implying significant computational and communication costs. Chattaraj et al. [22] proposed a certificateless AKA scheme based on blockchain. In this scheme, the key escrow problem is solved, and the computation and communication costs of the key agreement operation are guaranteed to be low. However, the scheme does not achieve decentralization in a real sense. Xu et al. [23] proposed an efficient AKA scheme based on blockchain, which uses only some lightweight computation with the assistance of blockchain to complete authentication and key agreement, reducing the computational costs of the whole authentication process. However, this scheme assumes that the TA is honest and trustworthy, resulting in the security assumption of this scheme being too high.

In summary, the schemes based on decentralized trusted architecture largely solve the problems of robustness, concurrency, security, and other aspects of the schemes based on centralized trusted architecture. However, the existing related research schemes have not considered how to construct efficient AKA

schemes using blockchain technology for VANETs semi-trusted environments, respectively, and they suffer from high computational and communication costs, low-security strengths, and high-security assumptions. Therefore, it is necessary to design a secure and efficient AKA scheme for non-fully trusted environments.

III. PRELIMINARIES AND BACKGROUND

A. Secret Sharing Schemes

A (t, n) secret sharing scheme over Z_q is a pair of efficient algorithms (G, C) that meet the following requirements [24].

- 1) G is a probabilistic algorithm that is invoked as $(\alpha_1, \dots, \alpha_s) \xleftarrow{R} G(s, t, \alpha)$, where $0 < t \leq s$ and $\alpha \in Z_q$, to generate a t -out-of- s sharing of α . It outputs s shares $SK := \{\alpha_1, \dots, \alpha_s\}$
- 2) C is a deterministic algorithm that is invoked as $\alpha \leftarrow C(\alpha'_1, \dots, \alpha'_t)$, to recover α .
- 3) Correctness: we require that for every $\alpha \in Z_q$, every set of s shares SK output by $G(s, t, \alpha)$, and every t -size subset $\{\alpha'_1, \dots, \alpha'_t\}$ of SK , we have that $C(\alpha'_1, \dots, \alpha'_t) = \alpha$.

B. Elliptic Curve Cryptosystem

Let F_p be a finite field, which is determined by a prime number p . We choose an elliptic curve $E: y^2 = x^3 + ax^2 + b \bmod p$ over the finite F_p , where $a, b \in F_p$. Let G denote a cyclic additive group of an elliptic curve over a finite field, $P \in G$ denote a generator of group, and q is the prime order of the group. Then we have the following computational or decisional hard problems.

- Elliptic curve discrete logarithm problem (ECDLP): Given two points P, Q on an elliptic curve E , which have the relation $Q = sP$, the probability of finding s for probabilistic polynomial-time (PPT) adversary is negligible.
- Elliptic curve computational Diffie-Hellman problem (ECDHP): Given three points P, xP, yP on an elliptic curve E , the probability of finding xyP for PPT adversary is negligible.
- Elliptic curve decisional Diffie-Hellman problem (ECDDHP): Given two tuples $\{P, xP, yP, xyP\}$ and $\{P, xP, yP, zP\}$ on an elliptic curve E , the probability of distinguishing two tuples for PPT adversary is negligible.

C. Cuckoo Filter

The Cuckoo filter [25] is a probabilistic data structure that employs fingerprint information (a short bit string derived from a hash function) for efficient storage and retrieval and supports deletion operation. Typically, a basic Cuckoo filter comprises a series of buckets, where each item has two candidate buckets determined by hash functions h_1 and h_2 .

Fig. 1 illustrates the insertion of a new item x into a Cuckoo filter with m buckets, where x 's fingerprint can be placed in the bucket $[i]$, where $i \in [0, m-1]$. If either of the two buckets for x is empty, the executor chooses one empty location to insert the fingerprint of x . However, if both buckets are occupied, the algorithm selects a candidate bucket and triggers a replacement operation. This replacement operation may involve recursion

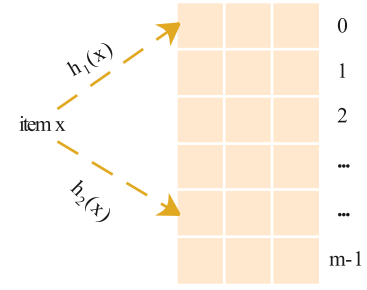


Fig. 1. The structure of cuckoo filter.

until it finds an empty location or reaches a specified replacement limit.

D. Blockchain Technology and Smart Contract

Blockchain technology is a distributed ledger technology that enables secure and transparent transactions between two or more parties. It is characterized by its decentralized and distributed ledger characteristics, which render the transaction process more secure, transparent, and free from interference from third parties.

A smart contract is a self-executing digital contract that is built on blockchain technology. It contains the terms and conditions of an agreement, written in code, and automatically executes actions when predefined conditions are met. Smart contracts eliminate the need for intermediaries, providing transparency, security, and efficiency in various transactions, such as financial transfers, asset exchanges, and more. They are tamper-proof and immutable, which makes transactions and contract execution more efficient, and reliable, and reduces costs and the risk of human error.

E. System Model

In our proposed scheme, the system model mainly includes four types of entities including the system initiator, edge server (ES), roadside unit (RSU), and vehicle, which are defined as follows and shown in Fig. 2.

- *System initiator*: The system initiator is mainly responsible for the generation of system and security parameters, the selection of edge servers that have the privilege of authenticating vehicles, and the management of the blockchain network. Note that once the system initiator is not involved in the AKA process.
- *ES*: The ES is considered as the semi-trusted network entity. In our proposed scheme, the ES is responsible for authenticating vehicles according to the credit of a specific vehicle. Additionally, the ES provides a registration service for vehicles in proximity.
- *RSU*: The RSU is mainly responsible for providing the network access service for vehicles. We assume that the RSU is equipped with an anomaly detection mechanism (such as the machine-learning-based anomaly detection mechanism) so that it can determine whether the communication behavior of a vehicle is malicious or not according to the historical communication behavior of the vehicle.

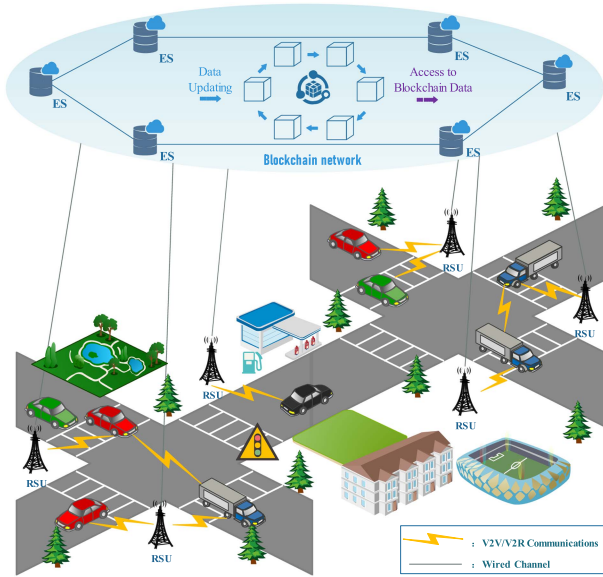


Fig. 2. System model.

- **Vehicle:** Vehicles are the most common communication entities in VANETs. Since vehicles are untrusted and the messages sent by vehicles are easy to be attacked, the legality of the vehicle and its messages that are involved in the AKA process need to be authenticated.

F. Security and Privacy Requirements

Given the reality that entities in VANETs communicate over insecure channels, a myriad of security challenges arise. Therefore, it becomes imperative to address the security and privacy requirements that the proposed scheme should satisfactorily fulfill.

- **Mutual authentication:** The entities involved in the key agreement process must achieve mutual authentication.
- **Confidentiality:** Only the vehicle and RSU that engage in the same AKA session have the capability to compute the session key correctly, which will be used for their subsequent communication.
- **Resistance to replay attacks:** An expired message cannot be exploited for launching a replay attack since it will not pass the authentication process.
- **Resistance to man-in-the-middle attacks:** If an adversary intercepts the message and attempts to replace or modify it, the message must not be successfully authenticated.
- **Resistance to DDoS attacks:** DDoS attacks launched against the network infrastructure do not affect the security of the entire system.
- **Resistance to collusion attacks:** The system must exhibit robustness capable of withstanding collusion attacks that are launched by the inner adversaries.
- **Resistance to key exposure attacks:** If the secret key held by the trust infrastructure is compromised or leaked, the system security should not be affected.

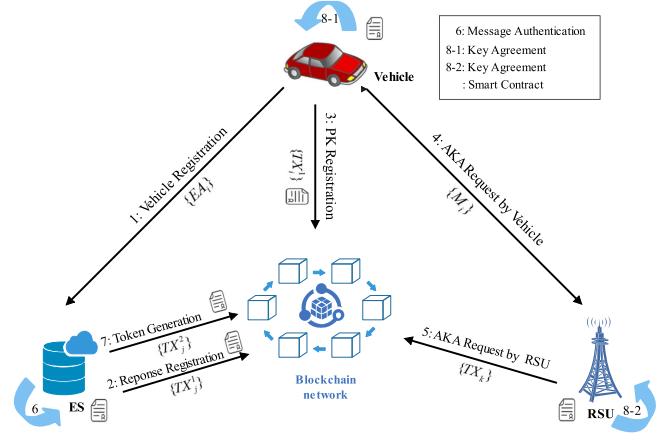


Fig. 3. The main process of the proposed scheme.

- **Forward and backward security:** Even if the session key for the current session is compromised, it poses no threat to the security of other session key agreement processes.
- **Perfect forward security:** Even if the long-term secret key is leaked or compromised, the session keys that were established in the older AKA sessions still cannot be recovered.
- **Conditional privacy-preserving:** The scheme will ensure the confidentiality of the vehicle's identity. However, if the vehicle behaves maliciously, then its real identity can be revealed by the official authority.

G. Threat Model

In this scheme, the entities in the consortium blockchain are considered to be semi-trusted, which jointly maintain the security of the blockchain network. The attackers in the scheme may be vehicles, RSUs, ESs, or other external attackers, and it is assumed that the number of ESs compromised by malicious adversaries does not exceed t . The type of attackers can be divided into internal and external attacks. Internal attacks may be initiated by registered nodes in the consortium blockchain to obtain secret information through conspiracy. An external attack is initiated by an external adversary, who may detect the channel in an attempt to obtain, modify, or fabricate the communication content.

IV. THE PROPOSED SCHEME

Our proposed scheme mainly contains four phases, i.e., setup phase, registration phase, request phase, and response phase, where the main interaction logic is as shown in Fig. 3.

A. Setup Phase

1) **Blockchain Construction:** The consortium blockchain is a specific blockchain with multiple management nodes to establish the distributed shared database with low confirmation delay. In our proposed scheme, RSUs and ESs, which are the most common semi-trusted entities of VANETs, are used to act as the management nodes of the consortium blockchain. In terms of specific management responsibilities, RSUs are mainly

responsible for evaluating the communication behavior of the vehicle, whereas ESs are mainly responsible for providing the registration and authentication service for vehicles.

2) *Parameter Initialization*: In this phase, the system initiator prepares all the necessary algorithms and parameters, such as ECDSA and hash functions. The RSUs in peer-to-peer mode use the consortium blockchain in a collaborative manner to complete the initialization in the absence of a trusted entity. In addition, the blockchain will also create a smart contract that manages the registered addresses.

- 1) The system initiator chooses a secure elliptic curve E : $y^2 = x^3 + ax + b \pmod{p}$. A point (or generator) P can be chosen from E to generate an additive elliptic curve group G with order q . Additionally, the system initiator chooses the system threshold t (the specific values depend on the balance between the system robustness and performance) that is used for the subsequent authentication process.
- 2) Assume that the number of ESs that can have the privilege to provide the registration and authentication services for vehicles is n , i.e., the authorized ES set will be $ES_j (1 \leq j \leq n)$. First, each ES_j applies to the system initiator to get the unique index $j (1 \leq j \leq n)$, generates two random secret values $u_j, e_j \in Z_q^*$, and computes its public keys $U_j = u_jP$ and $E_j = e_jP$ for achieving distributed authentication and secure transmission, respectively. Second, each ES_j generates a $(t-1)$ -order polynomial as shown in (1), where the coefficient $a_{j,1} \dots a_{j,t}$ are random numbers. Finally, each ES_j obtains its own secret sharing set ss_j according to (2).
- 3) Each ES_j in the authorized ES set uses the encrypt-then-sign paradigm (say ECIES with ECDSA) to get the authenticated encryption of $ss_{j,k} = (k, p_j(k))$ to other ESs , i.e., $ES_k (k \in [1, n] \& k \neq j)$, meaning that each ES_j can get secret sharing sets from other ESs securely.
- 4) All the public information that includes $\{E, G, q, P, n, U_j (1 \leq j \leq n), E_j (1 \leq j \leq n)\}$ will be stored in the genesis block of the consortium blockchain.

After the above operations are completed, each ES_j can calculate the cooperation private key sk_j by using the (3) according to the information on the chain; meanwhile, the system public key $PK = \sum_{j=1}^n U_j$ can be obtained.

$$p_j(x) = u_j + a_{j,1}x + a_{j,2}x^2 + \dots + a_{j,t}x^{t-1} \pmod{q} \quad (1)$$

$$ss_j = \{(idx, p_j(idx)) \mid idx \in [1, n] \& idx \neq j\} \quad (2)$$

$$sk_j = p_1(j) + p_2(j) + \dots + p_j(j) + \dots + p_n(j) \pmod{q} \quad (3)$$

B. Registration Phase

1) *ES and RSU Registration*: At first, each ES_j and RSU_k needs to generate the blockchain address $addr_j$ and $addr_k$ using the randomly generated numbers $s_j, s_k \in Z_q^*$, respectively, which will be written into the smart contract by the system initiator before the smart contract gets deployed, so that the sender address of the subsequent transaction used for authentication

Algorithm 1: Vehicle Registration Function.

Input: $addr_i$

Output: $bool$

- 1: $\triangleright$ check the validity of the function caller
 - 2: $\triangleright$ esa is an address array
 - 3: **if** $msg.sender$ **not in** esa **then**
 - 4: **return** $false$
 - 5: **else**
 - 6: $\triangleright va$ is a mapping type
 - 7: $va[msg.sender][addr_i] \leftarrow true$
 - 8: **return** $true$
 - 9: **end if**
-

should be equal to one of the valid addresses recorded in the blockchain.

2) *Vehicle Registration*: Since vehicles are considered untrusted communication authorities, the validity of the identities of vehicles needs to be checked, resulting in all vehicles needing to register with some ESs. To launch the registration request, the vehicle V_i chooses a random key $e_i \in Z_q^*$ to generate its blockchain address $addr_i$ and sends the corresponding ciphertext $EA_i = Enc(E_j, addr_i, id_i)$ to ES_j , where $Enc(pk, \cdot)$ and id_i denote the asymmetric encryption algorithm (say ECIES) and the real identity (say the license plate number) of V_i , respectively. Upon receiving the registration request from V_i , ES_j calls the smart contract function defined in Algorithm 1 by sending the signed transaction $TX_j^1 = \{addr_i\}_{e_j}$, so that the address $addr_i$ of V_i would be considered valid. Note that in Algorithm 1, the vehicle registration function can only be executed by ES, since this function would check the sender address (denoted by $msg.sender$) whether equal to one of address array (denoted by esa) that stores the blockchain addresses of all ESs.

3) *Public Key Registration*: Before sending the AKA request, the vehicle must register its public keys, which will be used in the authentication process. To register the public keys, V_i chooses a random number x_i and calculates $X_i = x_iP$. By repeating this procedure, V_i can generate a list of public keys denoted as pks_i . Then it will call the smart contract function defined in Algorithm 2 by sending the signed transaction $TX_i^1 = \{addr_j, pks_i\}_{e_i}$ to the blockchain network. Note that in Algorithm 2, the public keys of the vehicle would be registered with blockchain, so that the source credibility of the public keys can be guaranteed. Herein we use the Cuckoo filter to store the fingerprint of the public keys, thus greatly decreasing the storage costs of public keys since the size of the fingerprint is much lower than that of the public keys.

C. Request Phase

In this phase, the vehicle V_i first initiates an AKA request and sends the request to the roadside unit RSU_k for processing. Specifically, the phase contains the following steps.

1) *Vehicle Side*: First, V_i chooses a random number $q_i \in Z_q^*$ and calculates the corresponding public key $Q_i = q_iP$ for the subsequent key agreement. Then, V_i gets the current timestamp t_i and gets the corresponding signature ϵ_i as shown in (4), where

Algorithm 2: Public Key Registration Function.

Input: $addr_j, pks_i$
Output: *bool*

- 1: **if** $vf == \text{NULL}$ **then**
- 2: $\triangleright$ init an empty Cuckoo filter with specific table size and maximum kick counts
- 3: $vf \leftarrow \text{new}(\text{Size}, \text{MaxKicks})$
- 4: **end if**
- 5: **if** $va[addr_j][msg.sender] == \text{True}$ **then**
- 6: **for each** $pk_i \in pks_i$ **do**
- 7: $\triangleright$ call the insert function of the Cuckoo filter
- 8: $vf.insert(pk_i)$
- 9: **end for**
- 10: **return true**
- 11: **else**
- 12: **return false**
- 13: **end if**

X_i has been generated and registered in the registration phase. At last, V_i sends the message $M_i = \{addr_i, t_i, Q_i, X_i, \epsilon_i\}$ to RSU_k which is to be connected.

$$\epsilon_i = q_i + x_i \cdot h(addr_i, Q_i, t_i, X_i) \quad (4)$$

2) *RSU Side:* Upon receiving M_i , RSU_k checks the message freshness by determining whether the equation $|t_i - t_{cur}| \leq \Delta T$ holds, where t_{cur} denotes the current timestamp and ΔT denotes the max legal time interval. If not, RSU_k rejects this message and aborts the request. Otherwise, RSU_k chooses a random number $q_k \in Z_q^*$, calculates the corresponding public key $Q_k = q_k P$, gets the current timestamp t_k , generates the ECDSA-signed blockchain transaction $TX_k = \{addr_i, t_i, Q_i, X_i, \epsilon_i, Q_k, t_k\}_{s_k}$, and sends it to the consortium blockchain.

D. Response Phase

In this phase, the AKA request launched by V_i will be processed by ESs, who will generate signatures as the distributed authentication tokens for this request. If the number of these tokens reaches the threshold, the AKA request would be considered valid, which means that the participants who are involved in the AKA session can perform the key agreement process and establish the session key securely.

1) *Message Authentication:* Upon receiving TX_k , ES_j that is an active ES first checks the freshness of the timestamps t_k and t_i and aborts the process if either t_k or t_i fails to pass the freshness check. Second, ES_j checks the validity of transaction TX_k (by verifying the corresponding ECDSA signature of the transaction). Third, ES_j calls the smart contract function defined in Algorithm 3 to check the validity of X_i . If the function returns false, ES_j rejects this message and aborts the request since the public key or the ES identity of the message is considered invalid. Otherwise, ES_j checks the validity of the signature according to (5). If the equation does not hold, ES_j rejects this message and aborts the request since the message authentication does not

Algorithm 3: Public Key Verification Function.

Input: $h(pk_i)$
Output: *bool*

- 1: $\triangleright$ check whether pk_i exists in the filter
- 2: **if** $vf.lookup(h(pk_i)) == \text{true}$ **then**
- 3: **return true**
- 4: **else**
- 5: **return false**
- 6: **end if**

Algorithm 4: Token Aggregation Function.

Input: $h(TX_k), \sigma_j, R_j$
Output: *bool*

- 1: $\triangleright$ check whether the sender is an ES
- 2: **if** $msg.sender$ **not in** esa **then**
- 3: **return false**
- 4: **else**
- 5: $\triangleright count$ is used to store the token count.
- 6: $\triangleright token$ is used to store tokens.
- 7: $ctr = count[h(TX_k)]$
- 8: $tokens[h(TX_k)][ctr] \leftarrow (\sigma_j, R_j)$
- 9: $count[h(TX_k)] \leftarrow ctr + 1$
- 10: **return false**
- 11: **end if**

pass. Otherwise, the authentication process for the AKA request is done.

$$\epsilon_i \cdot P \stackrel{?}{=} Q_i + h(addr_i, Q_i, t_i, X_i) \cdot X_i \quad (5)$$

2) *Token Generation:* Once the authentication process for the AKA request has passed, ES_j can get one main conclusion, i.e., the AKA request message has not been modified or fabricated during the transmission process. However, the vehicle that launches the AKA request may be the malicious inside adversary, who may produce messages that can be authenticated but the content or behavior is malicious. To avoid this, ES_j can analyze the communication behavior of V_i according to the historical communication data or log with the assistance of nearby RSUs (we assume that ESs have been deployed with the anomaly detection platform). If V_i is considered as a legal vehicle from the view of ES_j , then ES_j can generate the token for V_i using (6), where x_k denotes the index k of ES_k (i.e., $x_k = k$ based on the index configure process in the Setup Phase). Finally, ES_j calls the smart contract function defined in Algorithm 4 by sending the signed transaction $TX_j^2 = \{h(TX_k), \sigma_j, R_j = r_j P\}_{s_j}$ to the blockchain.

$$\sigma_j = r_j + \left(sk_j \prod_{k \neq j}^{1 \leq k \leq t} \frac{0 - x_k}{x_j - x_k} \right) \cdot h(addr_i, addr_k, PK, h(TX_k)) \quad (6)$$

Algorithm 5: Token Validation Function.

Input: $h(TX_k)$
Output: $(token[])$
1: $ctr = count[h(TX_k)]$
2: **if** $ctr < t$ **then**
3: **return** NULL
4: **else**
5: ▷ return all of tokens related $h(TX_k)$.
6: **return** $tokens[h(TX_k)]$
7: **end if**

3) *Key Agreement*: Only if the number of tokens for TX_k generated by different ESs reaches the threshold t , the corresponding AKA request message will be considered authenticated. To authenticate the AKA request message, the AKA participants that include V_i and RSU_k can call the smart contract function defined in Algorithm 5, which will return a null flag if the threshold t has not been reached or return all tokens otherwise. If the number of the returned tokens is not null, i.e., the threshold t has been reached, then V_i or RSU_k can use (7) to verify the legality of TX_k . If the equation does not hold, the AKA request will be terminated. Otherwise, V_i or RSU_k can use (8) to establish the session key sk_{ik} securely, which can be used as the input of the key derivation function to generate a series of short-term session keys for achieve secure channel together the off-the-shelf encryption (say AES) and authentication functions (say MAC).

$$\begin{aligned} \sum_{j=1}^t \sigma_j \cdot P &= \sum_{j=1}^t R_j + \sum_{j=1}^t \left(sk_j \prod_{k \neq j}^{1 \leq k \leq t} \frac{0 - x_k}{x_j - x_k} \right) \cdot h(addr_i, \\ &\quad addr_k, PK, h(TX_k)) \cdot P \\ &= \sum_{j=1}^t R_j + h(addr_i, addr_k, PK, h(TX_k)) \cdot PK \end{aligned} \quad (7)$$

$$sk_{ik} = h(q_i \cdot Q_k, h(TX_k)) = h(q_k \cdot Q_i, h(TX_k)) \quad (8)$$

4) *Key Revocation*: To protect the privacy of the vehicles and to avoid the full load of the on-chain filter (causing some public keys to get kicked out), a key revocation operation will be performed after the end of each key agreement session. ESs will publish the public key information pks of the revoked vehicles invoking the smart contract to perform the $delete(pk)$ operation for each public key (as shown in Algorithm 6), and the updated Cuckoo filter will not contain the fingerprint of the revoked vehicle's public key (thus the revoked vehicle will not be able to pass the identity check).

V. SECURITY ANALYSIS

In this section, we'll prove the correctness of the authentication equation, outline the protocol's security model, prove its security within this framework, and analyze security and privacy

Algorithm 6: Public Key Revocation Function.

Input: pks
Output: $bool$
1: **if** $msg.sender$ **not in** esa **then**
2: **return** $false$
3: **else**
4: **for each** $pk \in pks$ **do**
5: $vf.delete(pk)$
6: **end for**
7: **end if**

requirements and the results from formal protocol verification tools.

A. Correctness

As described above, ESs needs to generate tokens for the AKA request transaction TX_k if the message and the corresponding sender both have been authenticated. The AKA participants, including V_i and RSU_k , can use (7) to check whether the requirement is satisfied or not. Herein we prove the correctness of (7) by employing the Lagrange interpolation theorem. The proof process is as shown in (9). Note that we use $h(M)$ to represent $h(addr_i, addr_k, PK, h(TX_k))$ in the following proof.

$$\begin{aligned} &\sum_{j=1}^t \sigma_j \cdot P \\ &= \sum_{j=1}^t r_j \cdot P + \sum_{j=1}^t \left(sk_j \prod_{k \neq j}^{1 \leq k \leq t} \frac{0 - x_k}{x_j - x_k} \right) \cdot h(M) \cdot P \\ &= \sum_{j=1}^t R_j + \left\{ (p_1(x_1) + p_2(x_1) + \dots + p_n(x_1) \bmod q) \cdot \right. \\ &\quad \prod_{k \neq 1}^{1 \leq k \leq t} \frac{0 - x_k}{x_1 - x_k} + (p_1(x_2) + p_2(x_2) + \dots + p_n(x_2) \\ &\quad \bmod q) \prod_{k \neq 2}^{1 \leq k \leq t} \frac{0 - x_k}{x_2 - x_k} + \dots + (p_1(x_t) + p_2(x_t) \\ &\quad \bmod q) \prod_{k \neq t}^{1 \leq k \leq t} \frac{0 - x_k}{x_t - x_k} \left. \right\} \cdot h(M) \cdot P \\ &= \left\{ \left(p_1(x_1) \prod_{k \neq 1}^{1 \leq k \leq t} \frac{0 - x_k}{x_1 - x_k} + p_1(x_2) \prod_{k \neq 2}^{1 \leq k \leq t} \frac{0 - x_k}{x_2 - x_k} + \dots \right. \right. \\ &\quad \left. \left. + p_1(x_t) \prod_{k \neq t}^{1 \leq k \leq t} \frac{0 - x_k}{x_t - x_k} \right) + \left(p_2(x_1) \prod_{k \neq 1}^{1 \leq k \leq t} \frac{0 - x_k}{x_1 - x_k} \right. \right. \\ &\quad \left. \left. + p_2(x_2) \prod_{k \neq 2}^{1 \leq k \leq t} \frac{0 - x_k}{x_2 - x_k} + \dots + p_2(x_t) \prod_{k \neq t}^{1 \leq k \leq t} \frac{0 - x_k}{x_t - x_k} \right) \right\} \end{aligned}$$

$$\begin{aligned}
& + \cdots + \left(p_n(x_1) \prod_{k \neq 1}^{1 \leq k \leq t} \frac{0 - x_k}{x_1 - x_k} + p_n(x_2) \prod_{k \neq 2}^{1 \leq k \leq t} \frac{0 - x_k}{x_2 - x_k} \right. \\
& \left. + \cdots + p_n(x_t) \prod_{k \neq t}^{1 \leq k \leq t} \frac{0 - x_k}{x_t - x_k} \right) \cdot h(M) \cdot P + \sum_{j=1}^t R_j \\
& = \left\{ \sum_{i=1}^t p_1(x_i) \prod_{k \neq i}^{1 \leq k \leq t} \frac{0 - x_k}{x_i - x_k} + \sum_{i=1}^t p_2(x_i) \prod_{k \neq i}^{1 \leq k \leq t} \frac{0 - x_k}{x_i - x_k} \right. \\
& \left. + \cdots + \sum_{i=1}^t p_n(x_i) \prod_{k \neq i}^{1 \leq k \leq t} \frac{0 - x_k}{x_i - x_k} \right\} \cdot h(M) \cdot P + \sum_{j=1}^t R_j \\
& = (u_1 + u_2 + \cdots + u_n) \cdot h(M) \cdot P + \sum_{j=1}^t R_j \\
& = \sum_{j=1}^t R_j + h(addr_i, addr_k, PK, h(TX_k)) \cdot PK \quad (9)
\end{aligned}$$

B. Security Model

First, we define a security model in which we use a series of games to show the behaviors of the adversary and the interactions between the adversary and the challenger. The participants are divided into three types, i.e., the vehicle V_i , roadside unit RSU_k , and edge server ES_j . We use Π_Λ^t to denote the t -th instance of participant $\Lambda \in \{V_i, RSU_k, ES_j\}$. In these games, adversary $\mathcal{A}$ can send queries to multiple oracles and will receive the corresponding response from challenger $\mathcal{C}$. The detailed oracles are defined as follows.

- 1) *Execute*(Π_V^i, Π_{RSU}^k): In this oracle, the challenger $\mathcal{C}$ will respond with the interaction message among instances Π_V^i and Π_{RSU}^k after receiving the query, meaning that the passive attacks can be simulated.
- 2) *Send*(Π_Λ^t, m_t): In this oracle, the challenger $\mathcal{C}$ will respond with the corresponding message according to the simulated protocol after receiving the request message m_t for the instance Π_Λ^t , meaning that the active attacks via modifying or fabricating messages can be simulated.
- 3) *Reveal*(Π_V^i, Π_{RSU}^k): In this oracle, the challenger $\mathcal{C}$ will respond with the session key that is established between Π_V^i and Π_{RSU}^k , meaning that the forward security can be simulated.
- 4) *Corrupt*(Π_Λ^t): In this oracle, the challenger $\mathcal{C}$ will respond with the long-term secret key of the instance Π_Λ^t (i.e., the secret key of the blockchain address), meaning that the forward security can be simulated.
- 5) *Test*(Π_V^i, Π_{RSU}^k): In this oracle, the challenger $\mathcal{C}$ will check whether the instances Π_V^i and Π_{RSU}^k have been requested in the last queries. If so, $\mathcal{C}$ will abort the request. Otherwise, $\mathcal{C}$ will toss a random coin $b \in \{0, 1\}$. If $b = 0$, $\mathcal{C}$ will choose a random key $sk_{ik} \in Z_q^*$. Otherwise, $\mathcal{C}$ will send the session key that was established by Π_V^i and Π_{RSU}^k to $\mathcal{A}$.

Definition 1 (Participants): If two instances Π_Λ^t and $\Pi_\Lambda^{t^*}$ are involved in the same AKA session, then Π_Λ^t and $\Pi_\Lambda^{t^*}$ can be considered as participants.

Definition 2 (Freshness): If neither the instance Π_Λ^t nor its participant $\Pi_\Lambda^{t^*}$ has requested *Reveal*($\cdot$) query, then we can consider that the freshness characteristic is satisfied for Π_Λ^t and $\Pi_\Lambda^{t^*}$.

Definition 3 (Correctness): If two instances Π_Λ^t and $\Pi_\Lambda^{t^*}$ that are involved in the same AKA session can calculate the common session key correctly, i.e., the independent calculation results are equal, then we can say that the correctness characteristic is satisfied.

Definition 4 (Session Key Semantic Security): To finish the game defined above, the adversary $\mathcal{A}$ is required to launch the *Test*($\cdot$) query and send the guess bit $\hat{b}$. If $\hat{b}$ equals b that is the actual choice of the challenger $\mathcal{C}$, then we say that $\mathcal{A}$ wins the game. The advantage that $\mathcal{A}$ wins the game is defined as the traditional semantic security definition, i.e., $Adv_{\mathcal{A}} = 2|Pr(W) - 1/2|$ where W denotes the winning event. The AKA protocol satisfies semantic security if $Adv_{\mathcal{A}}$ is negligible for any polynomial-time bounded probabilistic adversary.

C. Proven Security

Theorem 1: If any polynomial-time bounded probabilistic adversary cannot break ECDDHP, then the proposed AKA protocol satisfies session key semantic security under the above security model.

Proof: In this proof, we use the well-known game-hopping technique to construct a series of games, where the challenger interacts with the adversary according to the specific rules. We denote the event that $\mathcal{A}$ wins i -th game as W_i . And the detailed definition of Game i is defined as follows.

Game 1: In Game 1, the challenger $\mathcal{C}$ and adversary $\mathcal{A}$ follow the original protocol without modification. The challenger $\mathcal{C}$ is provided with a ECDDHP tuple $\{A = aP, B = bP, C = abP\}$ and $\{A = aP, B = bP, C = cP\}$ where a, b, c are random numbers in Z_q^* and uses $\mathcal{A}$ as a subroutine to break the ECDDHP tuple. Specifically, $\mathcal{A}$ can request the oracles defined as follows. It is worth noting that herein the interactions related to blockchain are simplified to the core cryptographic operations.

- 1) *Send*($\Pi_V^i, \Pi_{RSU}^k, reg$): Before processing all *Send*($\cdot$) queries, $\mathcal{C}$ initiates parameters according to the real scheme and key-value maps at first. Upon receiving this query, $\mathcal{C}$ checks whether $KV_1[\Pi_V^i]$ is null or not. If not null, $\mathcal{C}$ returns $KV_1[\Pi_V^i, \Pi_{RSU}^k]$ to $\mathcal{A}$. Otherwise, $\mathcal{C}$ generates $t_i, e_i, addr_i, X_i$, and TX_i^1 for Π_V^i and t_k, Q_k, TX_k for Π_{RSU}^k using the protocol method, stores $\{t_i, e_i, addr_i, X_i, TX_i^1, t_k, Q_k, TX_k\}$ as the value of $KV_1[\Pi_V^i, \Pi_{RSU}^k]$, and sends the value tuple to $\mathcal{A}$.
- 2) *Send*($\Pi_V^i, \Pi_{RSU}^k, \{X_i\}$): Upon receiving this query, $\mathcal{C}$ checks whether $KV_1[\Pi_V^i, \Pi_{RSU}^k]$ is empty or not. If so, $\mathcal{C}$ terminates this query. Otherwise, $\mathcal{C}$ extracts the value tuple from $KV_1[\Pi_V^i, \Pi_{RSU}^k]$, chooses $q_i, q_k \in Z_q^*$, calculates Q_i, Q_k , and ϵ_i using X_i , generates TX_k , and sends TX_k to $\mathcal{A}$.

- 3) *Send*($\Pi_V^i, \Pi_{RSU}^k, \{TX_k\}$): Upon receiving this query, $\mathcal{C}$ verifies the validity of TX_k using the verification algorithm of ECDSA and verification equation as shown in (5). If any of the verification fails, $\mathcal{C}$ terminates the check. Otherwise, $\mathcal{C}$ generates the tokens $\sigma_j (1 \leq j \leq t)$ using (6), stores $KV_2[\Pi_V^i, \Pi_{RSU}^k] = \sigma_j (1 \leq j \leq t)$ sends these tokens to $\mathcal{A}$. Note that the number of engaged ESs is t for simplicity.
- 4) *Send*($\Pi_V^i, \Pi_{RSU}^k, \{TX_k, \sigma_j (1 \leq j \leq t)\}$): Upon receiving this query, $\mathcal{C}$ checks the validity of tokens using (7). If not valid, $\mathcal{C}$ terminates this query. Otherwise, $\mathcal{C}$ calculates the session key sk_{ik} using (8), stores $KV_3[\Pi_V^i, \Pi_{RSU}^k] = sk_{ik}$, and sets the instances Π_V^i and Π_{RSU}^k as unfreshed instances.
- 5) *Execute*(Π_V^i, Π_{RSU}^k): Upon receiving this query, $\mathcal{C}$ extracts the corresponding values of $KV_1[\Pi_V^i, \Pi_{RSU}^k]$ and $KV_2[\Pi_V^i, \Pi_{RSU}^k]$. If one of the two values is empty, $\mathcal{C}$ terminates this query. Otherwise, $\mathcal{C}$ sends the two values to $\mathcal{A}$.
- 6) *Corrupt*(Π_V^i, Π_{RSU}^k): Upon receiving this query, $\mathcal{C}$ checks if $KV_3[\Pi_V^i, \Pi_{RSU}^k]$ is empty or not. If so, $\mathcal{C}$ terminates this query. Otherwise, $\mathcal{C}$ sends the value that is the established session key for Π_V^i and Π_{RSU}^k to $\mathcal{A}$.
- 7) *Test*(Π_V^i, Π_{RSU}^k): Upon receiving this query, $\mathcal{C}$ checks instances Π_V^i and Π_{RSU}^k are fresh or not. If not, $\mathcal{C}$ terminates this query. Otherwise, $\mathcal{C}$ tosses a random coin $b \in \{0, 1\}$. If $b = 1$, $\mathcal{C}$ extracts $sk_{ik} = KV_3[\Pi_V^i, \Pi_{RSU}^k]$ and sends sk_{ik} to $\mathcal{A}$. Otherwise, $\mathcal{C}$ chooses a random key that has the same length of sk_{ik} and sends it to $\mathcal{A}$.

It is obvious the above process is the same as the original protocol, resulting in (10) being satisfied.

$$Adv_{\mathcal{A}} = 2|Pr(W_1) - 1/2| \quad (10)$$

Game 2: In this game, all interactions are the same with Game 1 except that all hash functions are simulated as random oracles. Specifically, upon receiving the request m_i for the hash function $h(\cdot)$, $\mathcal{C}$ checks whether $h(m_i)$ is empty or not. If so, $\mathcal{C}$ chooses a random number r_i , sets $h(m_i) = r_i$, and sends $h(m_i)$ to $\mathcal{A}$. Otherwise, $\mathcal{C}$ returns $h(m_i)$ to $\mathcal{A}$ directly. It is obvious that Game 2 is indistinguishable from the view of $\mathcal{A}$, resulting in (11) being satisfied.

$$Pr(W_2) = Pr(W_1) \quad (11)$$

Game 3: In this game, all interactions are identical to Game 2 except that existential unforgeability under adaptive chosen message attacks (EU-CMA) are resisted. Before analyzing the winning advantage, we first prove that the authentication part of our proposed AKA scheme can be reduced to the hardness that breaks Schnorr signature. To prove it, we can construct the simulation where the number of involved ESs equals to t , i.e., the involved ESs are $ES_1, ES_2, \dots, ES_t$. Among these ESs, we assume that the adversary $\mathcal{A}$ controls $ES_1, \dots, ES_{t-1}$ whereas the forger $\mathcal{F}$ controls the honest participant ES_t (thus generality will not be lost). Based on the input of public key $PK = uP$ for the Schnorr signature algorithm, $\mathcal{F}$ interacts with $\mathcal{A}$ using the following simulations.

- 1) **Initialization:** a) $\mathcal{F}$ chooses a random $u_1 \in Z_q^*$, calculates $U_1 = u_1P$ and $p_1(x)$, and sends public material $\{(U_1, p_1(j)) | 2 \leq j \leq t\}$ to $\mathcal{A}$. b) $\mathcal{A}$ is required to send $\{(p_j(1), U_j) | 2 \leq j \leq t\}$ to $\mathcal{F}$. c) Based on the public key $PK = uP$, $\mathcal{F}$ rewinds $\mathcal{A}$ to step a) for replacing U_1 with $\hat{U}_1 = PK - \sum_{j=2}^t U_j$, so that the public material received by $\mathcal{A}$ is $\{\hat{U}_1, p_1(j) | 2 \leq j \leq t\}$.
- 2) **Token Generation:** a) For the token generation query for the AKA request message m , $\mathcal{F}$ chooses two random elements $R_1 \in \mathbb{G}$ and $\sigma_1 \in Z_q^*$ and sends them to $\mathcal{A}$. b) $\mathcal{A}$ is required to send $(R_j, \sigma_j) | 2 \leq j \leq t$ to $\mathcal{C}$. c) $\mathcal{C}$ sends m to the Schnorr signature oracle to get (σ, R) , calculates $\hat{R}_1 = R - \sum_{j=2}^t R_j$ and $\hat{\sigma}_1 = \sigma - \sum_{j=2}^t \sigma_j$, and rewinds $\mathcal{A}$ to step a) for replacing (R_1, σ_1) with $(\hat{R}_1, \hat{\sigma}_1)$.

In the above simulation, we can see that the produced tokens are valid since (12) holds, satisfying the verification method according to (7). Additionally, the difference between the real view and the simulated view is that we use $\hat{U}_1$ and $\hat{R}_1$ to replace U_1 and R_1 , respectively. It is obvious that these two generation methods are computationally indistinguishable from the view of $\mathcal{A}$. Therefore, we can construct a distinguisher that distinguishes Game 2 and Game 3 with the probability identical to breaking Schnorr and ECDSA (used for providing the integrity of the transaction message in the protocol) signature algorithms. According to the proofs in [26], [27], the advantages of breaking ECDSA and Schnorr signature algorithms for the PPT-adversary under the random oracle satisfy (13) and (14) where q_s denotes the request number of *Send*($\cdot$) oracle and ϵ denotes the probability of breaking ECDLP, respectively, resulting that (15) can be deduced.

$$\begin{aligned} \sum_{j=1}^t \sigma_j \cdot P &= \left(\hat{\sigma}_1 + \sum_{j=2}^t \sigma_j \right) \cdot P \\ &= \left(\sigma - \sum_{j=2}^t \sigma_j + \sum_{j=2}^t \sigma_j \right) \cdot P \\ &= \sigma \cdot P \\ &= R + h(PK, m) \cdot uP \\ &= \hat{R}_1 + \sum_{j=2}^t R_j + h(PK, m) \cdot PK \end{aligned} \quad (12)$$

$$\epsilon_1 \leq \frac{3q_s(q_s + q_h)}{(q-1)/2 - q_s - q_h} + (q_s + q_h)^2/2^q + q_h \epsilon \quad (13)$$

$$\epsilon_2 \leq \sqrt{\frac{q + 2q_s(q_s + q_h) + q(q_s + q_h)\epsilon}{q}} \quad (14)$$

$$|Pr(W_3) - Pr(W_2)| \leq \epsilon_1 + \epsilon_2 \quad (15)$$

Game 4: In this game, $\mathcal{C}$ chooses the random index $idx \in q_s$ and replaces Q_i, Q_k , and Q_{ik} with A, B , and C in the idx -th *Send*($\cdot$) oracle, respectively, resulting that We can construct a

distinguisher that can distinguish Game 3 and Game 4. If this distinguisher can distinguish Game 4 and Game 3, then $\mathcal{C}$ can use $\mathcal{A}$ to act as a subroutine to break ECDDHP. According to the analysis, it is obvious that (16) holds, where ϵ^* denotes the probability of breaking ECDDHP.

$$|Pr(W_4) - Pr(W_3)| \leq q_s \cdot \epsilon^* \quad (16)$$

In Game 4, $\mathcal{A}$ cannot distinguish a random key and sk_{ik} that is established using (8) with non-negligible probability unless $\mathcal{A}$ happens to guess $h(q_i \cdot Q_k, h(TX_k))$ successfully, and the probability of which reaches up to q_h/q . Therefore, we can deduce that (17) holds. By combining (10)–(17), we can get (18).

$$|Pr(W_4)| \leq \frac{1}{2} + q_h/q \quad (17)$$

$$\begin{aligned} Adv_{\mathcal{A}} &= 2|Pr(W_1) - 1/2| \\ &= 2|Pr(W_2) - 1/2| \\ &= 2|Pr[W_2] - Pr[W_3] + Pr[W_3] - Pr[W_4] \\ &\quad + Pr[W_4] - \frac{1}{2}| \\ &\leq 2(|Pr[W_3] - Pr[W_2]| + |Pr[W_4] - Pr[W_3]| \\ &\quad + |Pr[W_4] - \frac{1}{2}|) \\ &\leq 2(\epsilon_1 + \epsilon_2 + q_s \cdot \epsilon^* + q_h/q) \\ &\leq \frac{6q_s(q_s + q_h)}{(q-1)/2 - q_s - q_h} + 2(q_s + q_h)^2/2^q + 2q_h\epsilon \\ &\quad + 2\sqrt{\frac{q + 2q_s(q_s + q_h) + q(q_s + q_h)\epsilon}{q}} + 2q_s \\ &\quad \cdot \epsilon^* + 2q_h/q \end{aligned} \quad (18)$$

D. Security and Privacy Requirements

- 1) *Mutual authentication*: In the response phase, the ES authenticates the validity of transaction TX_k , X_i , and the signatures ϵ_i through the signature verification algorithm and (5), respectively. Simultaneously, to prevent AKA requests from vehicles vulnerable to internal attacks, ES analyzes the historical communication data or logs of V_i with the assistance of nearby RSUs for vehicle authentication. Upon successful vehicle authentication, ES_j generates an authentication token for TX_k using (6). When transaction TX_k acquires signature authentication tokens from t nodes originating from various ESs, both V_i and RSU_k can independently authenticate TX_k using (7). This authentication process guarantees the legitimacy and integrity of the AKA message, which has been proved in Theorem 1. Thus, the protocol achieves authentication for the transmitted message and its corresponding sender.
- 2) *Confidentiality*: The protocol generates the session keys $h(q_i \cdot Q_k, h(TX_k))$ and $h(q_k \cdot Q_i, h(TX_k))$, which can only be computed by V_i and RSU_k . As the private keys,

q_i and q_k , are required for session key generation and are exclusively held by the user, no other entity can calculate the session key except for V_i and RSU_k . Consequently, the protocol ensures confidentiality.

- 3) *Resistance to replay attacks*: In the scheme, timestamps t_i and t_k are added to the transaction for authentication, and the message receiver first checks the freshness of the timestamp and refuses to perform subsequent agreement requests upon receiving an expired message. Thus, the check of timestamps is able to resist replay attacks.
- 4) *Resistance to man-in-the-middle attacks*: Within this scheme, it is unfeasible for an adversary to execute a man-in-the-middle attack through the forgery of a signature transaction. Any alterations by an adversary to the registered account address, public or private key, or timestamp within the signature transaction would obstruct the authentication process and hinder the generation of the correct session key.
- 5) *Resistance to DDoS attacks*: In the event of an adversary initiating a DDoS attack to inject a substantial volume of messages into the VANETs, it can lead to delays in fulfilling legitimate requests. To counteract this, the scheme adds timestamps, enabling users to efficiently filter out a significant number of expired messages by assessing their freshness. In addition, the consortium blockchain in the scheme consists of multiple semi-trusted entities, which greatly improves the robustness of the system.
- 6) *Resistance to collusion attacks*: The adversary can collude with semi-trusted ESs for authentication, and according to the definition of the polynomial, it must control at least t semi-trusted ESs to make a fabricated AKA message pass the authentication, which is an infeasible scenario in the real VANETs provided that a suitable value t is set up.
- 7) *Resistance to key exposure attacks*: All ESs have the corresponding private key to generate the token. Even if the private key is compromised or leaked, the established session keys would not be affected since the private key is not involved in the key agreement process.
- 8) *Forward and backward secrecy*: The private keys used for key agreement in V_i and RSU_k , respectively, are randomly generated, so the private keys in different sessions are not correlated. It also means that the session keys generated in different sessions are not correlated, so the protocol has forward and backward secrecy.
- 9) *Perfect forward security*: In our proposed AKA scheme, the long-term session keys owned by the vehicles are only for generating messages that can be authenticated by the receivers. In other words, the long-term session keys are separated from the key agreement process. Therefore, the perfect forward security can be achieved.
- 10) *Conditional privacy-preserving*: The identity information of the vehicle is securely stored in the ciphertext EA_i , which cannot be decrypted even if the adversary steals the message sent by the vehicle. However, when the vehicle sends a malicious behavior, ES_j can decrypt EA_i to obtain the address information $addr_i$ and the

TABLE I
SECURITY AND PRIVACY COMPARISONS

	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11
Yang et al.'s scheme [28]	✓	✓	✓	✓	✓	✓	×	✓	✓	×	⊗
Vangala et al.'s scheme [6]	✓	✓	✓	✓	✓	×	✓	×	×	×	⊗
Ma et al.'s scheme [19]	✓	✓	✓	✓	✓	✓	×	×	×	×	⊗
Our proposed scheme	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	⊙

✓: The requirement is satisfied ⊗: Require fully-trusted environment
 ×: The requirement is not satisfied ⊙: Require semi-trusted environment

information id_i of the vehicle. Therefore, our proposed scheme achieves conditional privacy protection.

E. Security and Privacy Comparisons

In this section, we conduct a comparative analysis between our proposed scheme and several related schemes, including Yang et al.'s scheme [28], Vangala et al.'s scheme [6], and Ma et al.'s scheme [19] in terms of the satisfied security and privacy requirements. Let R1, R2, R3, R4, R5, R6, R7, R8, R9, R10, and R11 denote mutual authentication, confidentiality, resistance to replay attacks, resistance to man-in-the-middle attacks, forward and backward secrecy, resistance to DDoS attacks, conditional privacy-preserving, resistance to collusion attacks, perfect forward security, resistance to key exposure attacks, required trust level of servers. Table I presents a comprehensive comparison of security and privacy aspects. Our proposed scheme exhibits a higher degree of satisfaction for a greater number of security and privacy requirements.

F. Analysis With Verifpal

Verifpal is a cryptographic protocol validation tool for verifying the correctness of security protocols. It helps developers and security experts to check communication protocols such as authentication protocols, key exchange protocols, etc. To ensure their effectiveness and security in protecting user data and privacy. It is based on formal verification techniques and uses a methodology called Verifiable Security Protocol Analysis to automatically verify that a protocol meets certain security properties. These security properties can include authentication, confidentiality, integrity, and more.

We employed the Verifpal tool to verify the security and correctness of our proposed scheme. The information interaction in our scheme takes place mainly in the entities RSU, Vehicle, ES, and the blockchain network consisting of ESs. We use the Verifpal tool to verify the confidentiality and authenticity of the information interacting between the entities in our scheme. Specifically, we verified the encrypted message EA_i , the signed transactions TX_j^1 , TX_i^1 , ϵ_i , TX_k , TX_j^2 . As for the secret sharing scheme, we harnessed Verifpal-Shamir's SSS function to verify its reliability. This entailed the node ES_G generating three secret shares of the secret value s , which were securely distributed to the nodes ES_A and ES_B . In the authentication phase, the secret value s can be reconstructed based on information from any two ES nodes. Finally, we confirmed that both the RSU and the vehicle have successfully computed the session key for

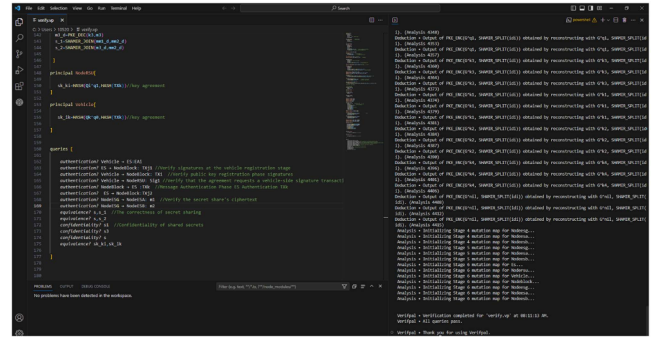


Fig. 4. Verifpal verification.

their subsequent communication. The results of our simulations using Verifpal are shown in Fig. 4

VI. PERFORMANCE ANALYSIS

In this section, we analyze the performance of our proposed scheme, where several tools were used for building the simulation environment. Specifically, the testbed is a PC that equips Intel i7-11700 K CPU together with 16 GB RAM. We use Veins framework that is based on the network simulation platform OMNeT++ [29] and traffic simulation software SUMO [30] for measuring the communication performance in VANETs environment, Miracl cryptography library [31] for measuring the execution time of cryptographic operations, and Ganache [32] together with Truffle framework that is a development environment, testing framework, and asset pipeline of Ethereum for measuring the blockchain-related performance.

A. Computational Costs Analysis

This section focuses on analyzing the computational costs of our proposed scheme and some other AKA schemes in VANETs including Yang et al.'s scheme [28], Vangala et al.'s scheme [6], and Ma et al.'s scheme [19]. We first introduce the representation and meaning of the computation time of each operation in the proposed scheme and other related schemes. We denote T_h , T_{aes} , T_{mtp} , T_p , T_{sm-bp} , T_{pa} , T_{sm-ecc} , T_{eca} , T_{bp} as hash function operation, AES encryption/decryption operation, map-to-point function operation, bilinear pairing operation, scalar multiplication operation related to bilinear pairing, point addition operation related to bilinear pairing, scalar multiplication operation related to elliptic curve, point addition operation related to elliptic curve, bivariate t -polynomial operation ($t = 10$). Table II

TABLE II
THE EXECUTION TIME OF INVOLVED CRYPTOGRAPHIC OPERATIONS

Operation	Definition	Symbolic description	Execution time (ms)
T_h	The execution time of hash function operation	$\{0, 1\}^* \rightarrow \mathbb{Z}_q^*$	0.002
T_{aes}	The execution time of AES encryption/decryption operation	$Enc_k(\cdot)$ or $Dec_k(\cdot)$	0.001
T_{mtp}	The execution time of map-to-point function operation	$\{0, 1\}^* \rightarrow \mathbb{G}_1$	0.061
T_p	The execution time of bilinear pairing operation	$e(A, B)$ where $A, B \in \mathbb{G}$	3.046
T_{sm-bp}	The execution time of scalar multiplication operation related to bilinear pairing	$x \cdot P$ where $P \in \mathbb{G}_1$	0.476
T_{pa}	The execution time of point addition operation related to bilinear pairing	$Enc_k(\cdot)$ or $Dec_k(\cdot)$	0.001
T_{sm-ec}	The execution time of scalar multiplication operation related to elliptic curve	$x \cdot P, i.e., P + P + \dots + P$	0.393
T_{eca}	The execution time of point addition operation related to elliptic curve	$P + P$ where $P \in \mathbb{G}_1$	0.002
T_{bp}	The execution time of bivariate t-polynomial operation ($t=10$)	$f(x, y) = \sum_{i,j=0}^t a_{ij}x^i y^j$	0.446

lists out the execution time. It is worth noting that we ignored some operations with negligible computational costs, such as Exclusive-OR operation.

In the Auth-I phase of Yang et al.'s scheme [28], to establish an authenticated session key, the vehicle must acquire the shares of at least t edge nodes to achieve authentication. After accomplishing the authentication, the vehicle and leader EN can get the session key computed by the hash function operation. The vehicle needs to perform $(t + 2)$ bilinear pairing operations, 3 scalar multiplication operations related to elliptic curve, 2 hash function operations, $(2t - 2)$ point addition operations related to elliptic curve, and $(t + 3)$ map-to-point function operations, so the computational cost of the vehicle is $(t + 2)T_p + 3T_{sm-ec} + (t + 3)T_{mtp} + (2t - 2)T_{eca} + 2T_h \approx (7.454 + 3.111t)$ ms, while an edge node needs to perform 4 bilinear pairing operations, 2 scalar multiplication operations related to elliptic curve, 1 hash function operation, and 4 map-to-point function operations, so the computational cost of an edge node is $4T_p + 2T_{sm-ec} + 4T_{mtp} + 1T_h \approx 13.261$ ms.

In the AKA phase of Vangala et al.'s scheme [6], both the vehicle and fog server need to perform 24 hash function operations, 10 scalar multiplication operations related to elliptic curve, and 4 point addition operations related to elliptic curve. So, the AKA-related computational cost of the vehicle or fog server is $24T_h + 10T_{sm-ec} + 4T_{eca} \approx 3.986$ ms.

In the AKA phase of Ma et al.'s scheme [19], the vehicle needs to perform 2 bivariate polynomial calculation operations, 5 scalar multiplication operations related to elliptic curve and 1 point addition operation related to elliptic curve, and 1 AES encryption operation, so the computational cost of the vehicle is $2T_{bp} + 5T_{sm-ec} + T_{eca} + T_{aes} \approx 2.86$ ms. The RSU needs to perform 2 bivariate polynomial calculation operations, 4 scalar multiplication operations related to elliptic curve and one point addition operation, and 1 AES decryption operation, so the computational cost of the RSU is $2T_{bp} + 4T_{sm-ec} + T_{eca} + T_{aes} \approx 2.467$ ms.

In the AKA phase of our proposed scheme, the vehicle needs to perform 6 scalar multiplication operations related to elliptic curve, $(t + 1)$ point addition operations related to elliptic curve, and 4 hash function operations, so the computational cost of the vehicle is $6T_{sm-ec} + (t + 1)T_{eca} + 4T_h \approx (2.368 + 0.002t)$ ms. The RSU needs to perform 7 scalar multiplication operations related to elliptic curve, $(t + 1)$

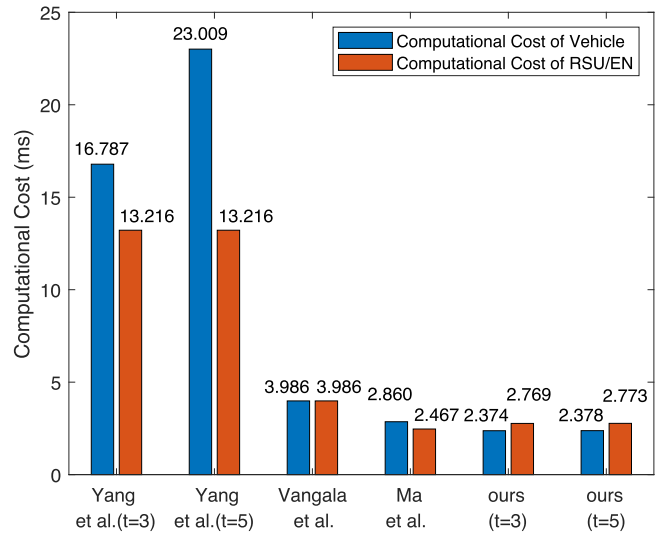


Fig. 5. The comparison of computational costs.

point addition operations related to elliptic curve, and 6 hash operations, so the computational cost of the RSU is $7T_{sm-ec} + (t + 1)T_{eca} + 5T_h \approx (2.763 + 0.002t)$ ms. Each ES needs to perform 4 scalar multiplication operations related to elliptic curve, 1 point addition operation related to elliptic curve, and 4 hash function operations, so the computational cost of an ES is $4T_{sm-ec} + T_{eca} + 4T_h \approx 1.582$ ms.

Based on the analysis presented above, the comparison of computational cost is as shown in Fig. 5. From Fig. 5, we can find that our proposed scheme outperforms all other compared schemes in terms of vehicle computational cost; Additionally, the computational cost incurred by RSU/EN is comparable to the Vangala et al.'s scheme [6] and Ma et al.'s scheme [19], which is obviously better than Yang et al.'s scheme [28], which indicates that our proposed scheme occupies less computational resources.

B. Communication Costs Analysis

This section focuses on analyzing the communication costs of our proposed scheme and some other AKA schemes in VANETs including Yang et al.'s scheme [28], Vangala et al.'s scheme [6], and Ma et al.'s scheme [19]. We set the 128-bit security level,

and we construct bilinear pairings on an elliptic curve $E : y^2 = x^3 + x$, where G is an additive group with order q and q is a 256-bit prime. At the same time, we choose an elliptic curve $E : y^2 = x^3 + ax + b \bmod p$, where p and the order q are two 256-bit primes and $a, b \in \mathbb{Z}_q$. Note that for the reasonableness of the comparison, we only consider the costs of the AKA phase.

In the AKA phase of Yang et al.'s scheme [28], in order to establish a session key between the vehicle and the leader EN . First, the vehicle needs to send $req = \{ID_v, Y, R_1, R_2, T_s\}$ to the leader $EN F_i$, where ID_v is the vehicle id , Y, R_1, R_2 are elements of G_1 , and T_s is a timestamp. Therefore, the communication cost related to AKA for the vehicle is $(512 * 3 + 256 + 32)/8 = 228$ bytes. Then the leader $EN F_i$ needs to send the req with EXP to the other EN s for authentication, where the $EXP \in \mathbb{Z}_q$. After receiving the messages from F_i , a set of at least $(t-1)$ EN s are ready to collaboratively authenticate the vehicle and return their V_j and W_j to F_i after determining the legality of the vehicle, where the V_j and W_j are elements of G_1 . Finally, The leader $EN F_i$ will send all the pairs of (V_k, W_k) and C to the vehicle, where $C \in \mathbb{Z}_q$. Therefore, the communication cost related to AKA for a leader $EN F_i$ or EN_j are $(512 * 3 + 256 * 3 + 32 + 512 * 2t)/8 = (292 + 128t)$ bytes and $(512+512)/8 = 128$ bytes.

In the MVFS AKA phase of Vangala et al.'s scheme [6], in order to establish a session key between the vehicle and the fog node. The vehicle needs to send the temporary identity $TID_M, \{U_M, TID_M, RID_M^*, Sig_M, K_M^h, TS_{mf1}\}$, and $\{SKV_{mf}, TS_{mf3}, TID_F^*\}$ to fog node FS , where $TID_M, SKV_{mf}, Sig_M, K_M^h, TID_F^*, RID_M^*$ are hash values, TS_{mf1}, TS_{mf3} are timestamps, and $U_M \in G$. Therefore, the communication cost of the vehicle for AKA is $(256 * 7 + 32 * 2 + 512)/8 = 296$ bytes. The fog node needs to send $\{TID_M, K_M^*, E_{Pub_M}(ID_M, K_{MV_i, FS_j}, RID_M, TS_{mc})\}$ and $\{V_F, Sig_F, TID_M^*, TID_F, RID_F^*, K_F^h, TS_{mf2}\}$ to vehicle, where $V_F \in G, TID_F, K_M^*, Sig_F, TID_M^*, RID_F^*, K_F^h$ are hash values, and TS_{mf2} is a timestamp. Therefore, the communication cost of the fog node for AKA is $(256 * 10 + 32 * 2 + 512 * 2)/8 = 392$ bytes.

In the AKA phase of Ma et al.'s scheme [19], in order to establish a session key between the vehicle and the RSU. The vehicle needs to send $\{ID_v, KV_v, M_{req}, T_v, HC_1, S_1\}$ to RSU for authentication, and $\{ID_v, HC_2, C_1, T_v\}$ for key agreement, where the ID_v is the vehicle id , KV_v is an element of GF_q , M_{req} is the request message, T_v is a timestamp, HC_1, HC_2 are hash values, C_1 is an ECIES encryption ciphertext of a hash value, and S_1 is an ECDSA signature. Therefore, the communication cost related to AKA for the vehicle is $(32 + 256 + 32 + 256 + 256 * 2 + 32 + 256 + 256 + 32)/8 = 208$ bytes. The RSU needs to send the message $\{ID_{RSU}, M_{res}, T_{RSU}, S_2\}$ and $\{ID_{rsu}, ID_v, HC_3, T_{RSU}\}$ to vehicle, where the M_{res} is the response message. Therefore, the communication cost related to AKA for RSU is $(32 + 32 + 256 * 2 + 32 + 256 + 32)/8 = 112$ bytes.

In the AKA phase of our proposed scheme, in order to establish a session key between the vehicle and the RSU. The vehicle needs to send $\{addr_i, t_i, Q_i, X_i, \epsilon_i\}$ to RSU for key

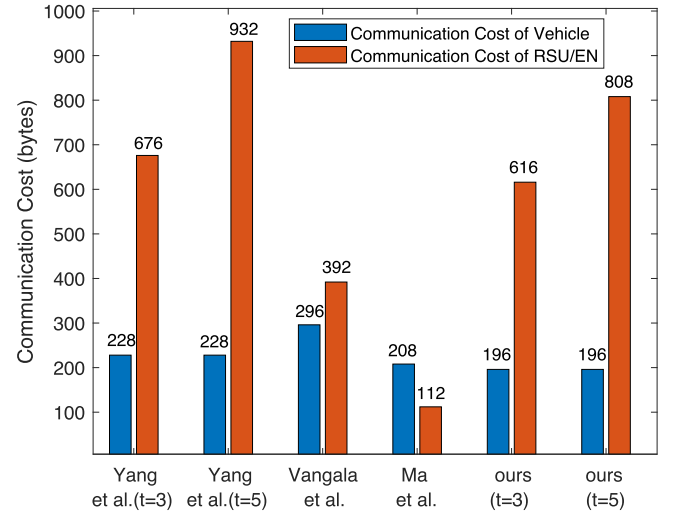


Fig. 6. The comparison of communication costs.

agreement, where the $addr_i$ is the vehicle address, t_i is a timestamp, Q_i and $X_i \in G$, and ϵ_i is the signature. Therefore, the communication cost related to AKA for the vehicle is $(256 * 2 + 512 * 2 + 32)/8 = 196$ bytes. The RSU needs to send $\{addr_i, t_i, Q_i, X_i, \epsilon_i, Q_k, t_k\}$ that is an ECDSA-signed transaction TX_k to blockchain for authentication, where t_k is also a timestamp. Additionally, the vehicle needs to extract the tokens via Algorithm 5, resulting in the RSU needing to send the requested t tokens $\{h(TX_k), \sigma_j\}$ together with the corresponding ECDSA signature to the vehicle. Therefore, the communication cost related to AKA for the RSU is $(256 * 2 + 32 * 2 + 512 * 4 + (256 + 512)t)/8 = 328 + 96t$ bytes.

Based on the analysis presented above, the comparison of communication cost is as shown in Fig. 6. From Fig. 6, we can find that our proposed scheme outperforms all other compared schemes in terms of vehicle communication costs; Additionally, the communication costs incurred by RSU/EN are comparable to that of the other compared schemes, suggesting that our proposed scheme holds certain advantages in terms of transmission bandwidth.

C. Network Simulation Analysis

To analyze the real performance of our proposed scheme, we use Veins 5.2 framework that is a middleware between OMNeT++ and SUMO to simulate our proposed scheme. In our simulation, we set up an environment based on a real-world map with 2.5 square kilometers. One hundred vehicles communicate with two RSUs using IEEE 802.11p. Specifically, every vehicle sends the AKA request message to RSU in a random interval that is slightly higher than one second (so frequent but random AKA requests can be simulated) and RSU would send the unicast communication.

We conduct an analysis of the AKA delay for these schemes. The AKA delay, in this context, is defined as the cumulative sum of the transmission delay and computational delay associated with AKA operations. It can be computed using (19), where M

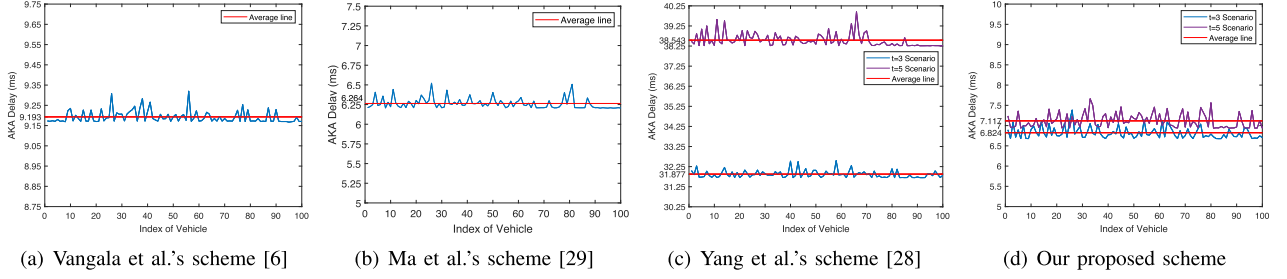


Fig. 7. The average AKA delays of our proposed and comparison schemes.

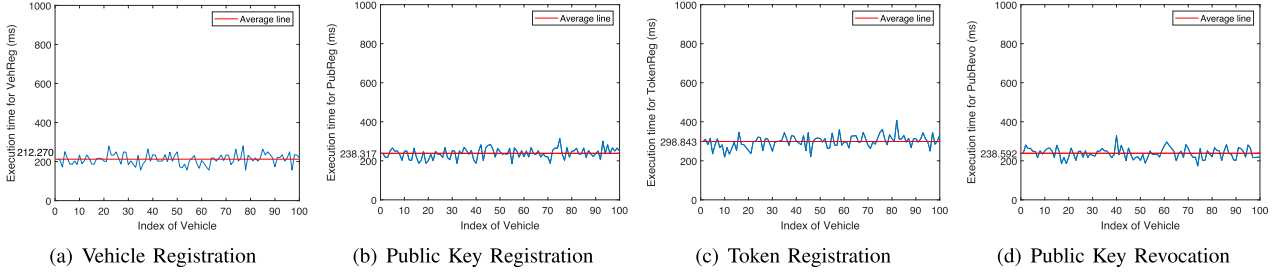


Fig. 8. The processing delays for state-changing blockchain operations.

represents the number of vehicles participating in AKA sessions, m_j signifies the number of AKA sessions for vehicle V_j , t_{end}^k denotes the message receiving time for session k , t_{beg}^k indicates the request initiation time for session k , and t_{aka} accounts for the computational time required for AKA-related operations. The simulation results portraying the average AKA delays are visually presented in Fig. 7. This depiction showcases a notable advantage of our proposed scheme significantly lower AKA delays in comparison to other AKA schemes.

$$AKADelay = \frac{1}{M} \sum_{j=1}^M \left(\frac{1}{m_j} \sum_{k=1}^{m_j} (t_{beg}^k - t_{end}^k + t_{aka}) \right) \quad (19)$$

D. Blockchain-Related Costs Analysis

Since our proposed AKA scheme relies on blockchain, we used Ganache to build a consortium blockchain network, implemented the smart contract using Solidity programming language, and used Truffle framework to measure the real performance. Our evaluation includes two types, i.e., gas costs and processing delays, where gas costs refer to the gas used for the state-changing blockchain operations, and processing delay refers to the distance between the request launching time and responding time.

As for the gas costs, we use the Web3 callback function of the transaction receipt to measure the gas costs, where the collection results are as shown in Table III. The gas cost for filter initiation operation in Table III is represented with a bivariate linear equation in which i and b denote the bucket number of the filter and the slot number of one bucket, respectively. The filter initiation operation is executed subsequently after the contract deployment

TABLE III
THE GAS COSTS OF STATE-CHANGING OPERATIONS

Operations	Gas
Contract Deployment	1576698
Filter Initiation	$152748 + 80973 * i + 4811.9 * b$
Vehicle Registration	44937
Public Key Registration	59535
Token Registration	108139
Public Key Revocation	28236

for building the initial filter or adjusting the capacity during the contract running process. Note that the bivariate linear equation is generated using the multiple linear regression method based on our experiment, which sets up multiple combinations of different i and b .

As for the processing delays, we built an experiment that simulates 100 vehicles sending different transactions to trigger the executions of state-changing blockchain operations, where the results can be shown in Fig. 8. From Fig. 8, we can see that the processing delay has certain fluctuations because of the task scheduling mechanism for operating systems and the transaction processing variety of the blockchain network. Additionally, the average processing delays for vehicle registration, public key registration, token registration, and public key revocation operations are 212.270 ms, 238.317 ms, 298.843 ms, and 238.592 ms, respectively.

According to the simulation experiments, the processing delay in our proposed scheme adheres to the specified maximum delay standard (500 ms) for V2X communication within the high latency, low-frequency category of applications outlined in T/CSAE 53-2020 [33]. Additionally, considering the infrequent

utilization of AKA in real-world scenarios, the natural gas costs in our scheme are deemed reasonable. Therefore, it can be concluded that our scheme can meet the requirements of most applications in the VANETs.

VII. CONCLUSION

Herein we have proposed a threshold-based full-decentralized AKA scheme for VANETs powered by consortium blockchain. In this scheme, semi-trusted ESs are used as the trust infrastructure to provide the authentication service, freeing the reliance on the ideal full-trust environment. The design leverages the concept of threshold-based voting, ensuring that the security of the system is not affected by certain violated ESs, thus enhancing the overall security. Additionally, we have designed a distributed authentication mechanism based on the Cuckoo filter and smart contract. This mechanism ensures the authenticity of AKA messages while implementing public key registration, validation, and revocation functions with low on-chain operation costs. Furthermore, our scheme includes rigorous proof of correctness, security, and adherence to fundamental security and privacy requirements. We also validate the interaction security within the protocol using the formal verification tool Verifpal. Finally, a performance analysis demonstrates the advantages of our proposed scheme in terms of computation costs, communication costs, and simulation performance.

ACKNOWLEDGMENT

The authors are very grateful to the anonymous referees for their detailed comments and suggestions regarding this paper.

REFERENCES

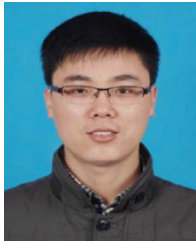
- [1] P. Vijayakumar, M. Azees, S. A. Kozlov, and J. J. Rodrigues, "An anonymous batch authentication and key exchange protocols for 6G enabled VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1630–1638, Feb. 2022.
- [2] A. Dua, N. Kumar, A. K. Das, and W. Susilo, "Secure message communication protocol among vehicles in smart city," *IEEE Trans. Veh. Technol.*, vol. 67, no. 5, pp. 4359–4373, May 2018.
- [3] M. A. Saleem, X. Li, M. F. Ayub, S. Shamshad, F. Wu, and H. Abbas, "An efficient and physically secure privacy-preserving key-agreement protocol for vehicular ad-hoc network," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 9, pp. 9940–9951, Sep. 2023.
- [4] L. Wei, J. Cui, H. Zhong, I. Bolodurina, and L. Liu, "A lightweight and conditional privacy-preserving authenticated key agreement scheme with multi-TA model for fog-based VANETs," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 1, pp. 422–436, Jan./Feb. 2023.
- [5] X. Li, J. Liu, M. S. Obaidat, P. Vijayakumar, Q. Jiang, and R. Amin, "An unlinkable authenticated key agreement with collusion resistant for VANETs," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 7992–8006, Aug. 2021.
- [6] A. Vangala, A. K. Das, A. Mitra, S. K. Das, and Y. Park, "Blockchain-enabled authenticated key agreement scheme for mobile vehicles-assisted precision agricultural IoT networks," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 904–919, Dec. 2023.
- [7] Y. Liu, Y. Wang, and G. Chang, "Efficient privacy-preserving dual authentication and key agreement scheme for secure V2V communications in an IoV paradigm," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 10, pp. 2740–2749, Oct. 2017.
- [8] J.-L. Huang, L.-Y. Yeh, and H.-Y. Chien, "ABAKA: An anonymous batch authenticated and key agreement scheme for value-added services in vehicular ad hoc networks," *IEEE Trans. Veh. Technol.*, vol. 60, no. 1, pp. 248–262, Jan. 2011.
- [9] X. Wang, Z. Huang, Q. Wen, and H. Zhang, "An efficient anonymous batch authenticated and key agreement scheme using self-certified public keys in VANETs," in *Proc. IEEE Int. Conf. IEEE Region 10*, 2013, pp. 1–4.
- [10] Q. Jiang, J. Ni, J. Ma, L. Yang, and X. Shen, "Integrated authentication and key agreement framework for vehicular cloud computing," *IEEE Netw.*, vol. 32, no. 3, pp. 28–35, May/Jun. 2018.
- [11] J. Cui, Y. Wang, J. Zhang, Y. Xu, and H. Zhong, "Full session key agreement scheme based on chaotic map in vehicular ad hoc networks," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 8914–8924, Aug. 2020.
- [12] S. H. Islam, M. S. Obaidat, P. Vijayakumar, E. Abdulhay, F. Li, and M. K. C. Reddy, "A robust and efficient password-based conditional privacy preserving authentication and group-key agreement protocol for VANETs," *Future Gener. Comput. Syst.*, vol. 84, pp. 216–227, 2018.
- [13] J. Cui, X. Tao, J. Zhang, Y. Xu, and H. Zhong, "HCPA-GKA: A hash function-based conditional privacy-preserving authentication and group-key agreement scheme for VANETs," *Veh. Commun.*, vol. 14, pp. 15–25, 2018.
- [14] L. Zhang, Q. Wu, J. Domingo-Ferrer, B. Qin, and C. Hu, "Distributed aggregate privacy-preserving authentication in VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 18, no. 3, pp. 516–526, Mar. 2017.
- [15] S. Son, J. Lee, Y. Park, Y. Park, and A. K. Das, "Design of blockchain-based lightweight V2I handover authentication protocol for VANET," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 3, pp. 1346–1358, May/Jun. 2022.
- [16] C. Lin, X. Huang, and D. He, "EBCPA: Efficient blockchain-based conditional privacy-preserving authentication for VANETs," *IEEE Trans. Dependable Secure Comput.*, vol. 20, no. 3, pp. 1818–1832, May/Jun. 2023.
- [17] F. Li, Z. Guo, C. Zhang, W. Li, and Y. Wang, "ATM: An active-detection trust mechanism for VANETs based on blockchain," *IEEE Trans. Veh. Technol.*, vol. 70, no. 5, pp. 4011–4021, May 2021.
- [18] H. Feng, D. Chen, and Z. Lv, "Blockchain in digital twins-based vehicle management in VANETs," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 10, pp. 19 613–19 623, Oct. 2022.
- [19] Z. Ma, J. Zhang, Y. Guo, Y. Liu, X. Liu, and W. He, "An efficient decentralized key management mechanism for VANET with blockchain," *IEEE Trans. Veh. Technol.*, vol. 69, no. 6, pp. 5836–5849, Jun. 2020.
- [20] Q. Zhang et al., "Blockchain-based asymmetric group key agreement protocol for Internet of Vehicles," *Comput. Elect. Eng.*, vol. 86, 2020, Art. no. 106713.
- [21] C. Meng, H. Zhang, H. Ji, and X. Li, "Mutual authentication and distributed key management with permissioned blockchain in MEC-enabled vehicular networks," in *Proc. 7th IEEE Int. Conf. Netw. Intell. Digit. Content*, 2021, pp. 393–397.
- [22] D. Chattaraj, B. Bera, A. K. Das, S. Saha, P. Lorenz, and Y. Park, "Block-CLAP: Blockchain-assisted certificateless key agreement protocol for Internet of Vehicles in smart transportation," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 8092–8107, Aug. 2021.
- [23] Z. Xu, W. Liang, K.-C. Li, J. Xu, and H. Jin, "A blockchain-based roadside unit-assisted authentication and key agreement protocol for Internet of Vehicles," *J. Parallel Distrib. Comput.*, vol. 149, pp. 29–39, 2021.
- [24] D. Boneh and V. Shoup, "A graduate course in applied cryptography," 2020. [Online]. Available: <http://toc.cryptobook.us>
- [25] B. Fan, D. G. Andersen, M. Kaminsky, and M. D. Mitzenmacher, "Cuckoo filter: Practically better than bloom," in *Proc. 10th ACM Int. Conf. Emerg. Netw. Experiments Technol.*, 2014, pp. 75–88.
- [26] M. Ferschi, E. Kiltz, and B. Poettering, "On the provable security of (EC) DSA signatures," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 1651–1662.
- [27] D. Pointcheval and J. Stern, "Security arguments for digital signatures and blind signatures," *J. Cryptol.*, vol. 13, pp. 361–396, 2000.
- [28] A. Yang, J. Weng, K. Yang, C. Huang, and X. Shen, "Delegating authentication to edge: A decentralized authentication architecture for vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1284–1298, Feb. 2022.
- [29] OMNeT++ discrete event simulator, 2023. Accessed: Jul. 29, 2023. [Online]. Available: <https://omnetpp.org/>
- [30] Simulation of urban mobility, 2023. Accessed: Jul. 29, 2023. [Online]. Available: <https://sumo.dlr.de/docs/index.html>
- [31] MIRACL cryptographic SDK, 2023. Accessed: Jul. 29, 2023. [Online]. Available: <https://github.com/miracl/MIRACL/>
- [32] Ganache v2.6.0: A personal ethereum blockchain, 2023. Accessed: Jul. 29, 2023. [Online]. Available: <https://trufflesuite.com/ganache/>
- [33] T. Society of Automotive Engineers of China, "Cooperative intelligent transportation system — vehicle communication application layer specification and data exchange standard (phase I)," *Cooperative Intell. Transp. Syst.*, Dec. 2020. [Online]. Available: <https://www.codeofchina.com/standard/TCSAE53-2020.html>



Lu Wei is currently a lecturer with the School of Computer Science and Technology, Anhui University. His research interests include security and privacy issues in vehicular ad hoc networks, applied cryptography, and blockchain. He has more than 10 scientific publications in reputable journals (e.g., *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Mobile Computing*, and *IEEE Transactions on Intelligent Transportation Systems*).



Yongjuan Zhang is currently a research student with the School of Computer Science and Technology, Anhui University. Her research focuses on the security of the vehicular ad hoc networks.



Jie Cui (Senior Member, IEEE) received the PhD degree from the University of Science and Technology of China, in 2012. He is currently a professor and PhD supervisor with the School of Computer Science and Technology, Anhui University. His current research interests include applied cryptography, IoT security, vehicular ad hoc network, cloud computing security, and software-defined networking (SDN). He has more than 150 scientific publications in reputable journals (e.g., *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Computers*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Industrial Electronics*, *IEEE Transactions on Cloud Computing* and *IEEE Transactions on Multimedia*), academic books and international conferences. He is in the Editorial Board of several international journals, such as the *IET Communications*, *Security and Communication Networks*, and *Sensors*.



Hong Zhong received the PhD degree in computer science from the University of Science and Technology of China, in 2005. She is currently a professor and PhD supervisor with the School of Computer Science and Technology, Anhui University. Her research interests include applied cryptography, IoT security, vehicular ad hoc network, cloud computing security, and software-defined networking (SDN). She has more than 200 scientific publications in reputable journals (e.g., the *IEEE Journal on Selected Areas in Communications*, *IEEE Transactions on Parallel and Distributed Systems*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Multimedia*, *IEEE Transactions on Vehicular Technology*, *IEEE Transactions on Network and Service Management*, *IEEE Transactions on Cloud Computing*, *IEEE Transactions on Industrial Informatics*, *IEEE Transactions on Industrial Electronics* and *IEEE Transactions on Big Data*), academic books and international conferences.



Irina Bolodurina received the PhD degree from South Ural State University. She is currently a professor and head with the Department of Applied Mathematics, Orenburg State University. She has more than 60 scientific publications in academic journals and international conferences indexed in Scopus and WoS. She has participated in more than 20 scientific projects supported by the RFBR and other Russian scientific programs. Her current research interests include theory of optimal control, mathematical modeling, information analysis software, control of social and economic systems, decision support systems, data integration, and processing.



Debiao He received the PhD degree in applied mathematics from the School of Mathematics and Statistics, Wuhan University, Wuhan, China, in 2009. He is currently a professor with the School of Cyber Science and Engineering, Wuhan University, and the Shanghai Key Laboratory of Privacy Preserving Computation, MatrixElements Technologies, Shanghai, China. His main research interests include cryptography and information security, in particular, cryptographic protocols. He has published more than 100 research papers in refereed international journals and conferences, such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Security and Forensic*, and *Usenix Security Symposium*. He is the recipient of the 2018 IEEE Sysems Journal Best Paper Award and the 2019 IET Information Security Best Paper Award. His work has been cited more than 10,000 times with Google Scholar. He is in the editorial board of several international journals, such as the *Journal of Information Security and Applications*, *Frontiers of Computer Science*, and *Human-centric Computing and Information Sciences*.